

Sabancı University

Faculty of Engineering and Natural Sciences

Computer Science and Engineering

PixelProof AI Image Detection and Reliability Evaluation

CS395 Internship Final Report

Efe Han Keleş

Student ID: 31994

Company: Türk Telekomünikasyon A.Ş.

Supervisor: Önder Çelebi

Supervisor title: Director

Internship: 20 July–18 September 2026

Approved duration: 40 internship days

Format: Hybrid

Submission date: [TO COMPLETE]

Abstract

Türk Telekom provides telecommunications and digital services across Türkiye. During my CS395 internship, I developed PixelProof to investigate AI-generated image detection while reducing false alerts on real photographs. A second experimental module explored locating AI-edited regions. Development followed an iterative process: inspect errors, define the next experiment, measure the result and record the decision. Dataset records tracked acquisition and permitted use, while experiment and history logs retained both improvements and failed approaches. Early transfer failures and label errors shifted the focus toward data quality and reliable evaluation. E92's learned components used 12,269 training images represented under four processing conditions, producing 49,076 training views. A separate but repeatedly used development collection contained 160 real and 160 AI images. E92 detected 159 AI images in each of two evaluation conditions. Compared with E43, false alerts fell from 68 to zero on real originals and from 68 to 14 after resizing and JPEG compression. All 20 numerical checks passed, but one newly missed reference-detected AI image prevented full acceptance. The deliverables include trained research components, a local web demonstration and reproducible records. Model2 remains experimental. The project demonstrates improvement on a defined development collection, rather than universal detection or production readiness. Further work should use independent, diverse evaluation sources, keep calibration separate from evaluation and test localization with more editors.

Table of Contents

1. Introduction.....	5
2. Company information.....	6
2.1 Corporate profile and services.....	6
2.2 Organization and internship context.....	7
3. Project background.....	8
3.1 Department information.....	8
3.2 Initial project status.....	8
3.3 Motivation and problem definition.....	9
3.4 Related literature.....	9
4. Internship project.....	11
4.1 Project objective and scope.....	11
4.2 My responsibilities.....	11
4.3 Methodology and tools.....	12
4.4 Expected outcomes and deliverables.....	13
4.5 Project details.....	13
4.5.1 Early models and the first transfer failures.....	13
4.5.2 Data integrity and corrected conclusions.....	14
4.5.3 Data roles and processing conditions.....	15
4.5.4 E92 representation and learning.....	16
4.5.5 Development improvement and acceptance limits.....	18
4.5.6 Honest scores and the web demonstration.....	19
4.5.7 Broader checks and rejected improvements.....	20
4.5.8 Model2 localization and its limits.....	21

Table of Contents continued

4.6 Results.....	22
5. Internship experience.....	23
5.1 Learning.....	23
5.2 Relation to undergraduate education.....	23
5.3 Major difficulties.....	24
5.4 A typical day.....	24
6. Conclusions.....	25
7. Recommendations.....	26
8. References.....	27
9. Appendices.....	29
9.1 Development criteria.....	29
9.2 Evidence and reproduction map.....	30

1. Introduction

PixelProof addresses a practical question: can a system identify evidence of AI generation without routinely accusing genuine photographs? Visually convincing generated images make this task difficult, while resizing and compression can change the signals available to a detector. The project also explored a related question: can a model locate a small AI-edited region inside an otherwise genuine photograph? These are different tasks and require different evidence.

The internship ran from 20 July to 18 September 2026 under the approved 40-day CS395 arrangement at Türk Telekom. This report covers the technical record through 16 September, the latest completed research checkpoint before the end of that period. Report preparation took place afterwards. Experiment identifiers refer to recorded protocols and findings, rather than a count of independent trials. The implementation and append-only records provide the primary evidence for project-specific claims (Keleş, 2026).

The main achievement is a functioning research system with measurable development improvements, a usable local interface and an evaluation process that also records failures. The current local demonstration uses E92. It passes 20 numerical checks on reused development images, but newly misses one AI image caught by the reference model. That new miss breaks a separate acceptance rule. Independent universal reliability remains unproven.

Section 2 introduces the host organization. Section 3 explains the problem and relevant literature. Section 4 describes responsibilities, methods, implementation, representative experiments and results. Section 5 discusses learning and difficulties. Sections 6 and 7 give conclusions and recommendations. References and a compact evidence appendix support the main account without reproducing every experiment or implementation detail.

2. Company information

2.1 Corporate profile and services

The host organization is Türk Telekomünikasyon A.Ş. Its registered office is Turgut Özal Bulvarı, 06103 Aydınlıkevler, Ankara (Türk Telekom, 2026a). I completed my internship at Fatih Sultan Mehmet Mah., Balkan Cad. No:49, 34771 Ümraniye, İstanbul. This is the internship office, distinct from the registered headquarters. The corporate website is <https://www.turktelekom.com.tr>. The internship contact supplied in the approved record is +90 312 555 93 92.

Türk Telekom has a history extending over 185 years. It adopted an integrated operating structure in 2015 and brought mobile, internet, telephone and television services under one brand in 2016. Its facilities are communications infrastructure, service operations and offices rather than a manufacturing plant. The corporate profile reports operations in 81 provinces and 31,756 employees alongside its 30 June 2026 operating figures (Türk Telekom, 2026b).

The group supplies fixed access, broadband, mobile, television and corporate data services. Customers include households, businesses and wholesale operators. Group companies include TT Mobil, TTNET, Argela, Innova, SEBİT, AssisTT and Türk Telekom International. The disclosed ownership structure allocates 60% to Türkiye Wealth Fund, 25% to the Ministry of Treasury and Finance and 15% to publicly traded shares. The fund also holds 1.68% within the publicly traded portion; that holding must not be added to the 100% ownership total (Türk Telekom, 2026a, 2026b).

The company operates in the telecommunications market, with competition across mobile and fixed connectivity. Competitors and suppliers were mentioned during my internship, but we did not discuss them in detail. The main competitors and major suppliers relevant to the unit remain [TO COMPLETE: confirmed competitor and supplier names]. I have not inferred

a supplier ranking from these brief discussions. PixelProof used research datasets and public pretrained components, not customer telecommunications records.

2.2 Organization and internship context

Figure 1 summarizes the corporate reporting structure published in the 2025 Annual Report. It retains the distinction between functions reporting to the general manager and internal audit reporting to the board. Functional descriptions are summarized from the published department titles. The diagram provides corporate context; the reporting line of my host unit still requires confirmation (Türk Telekom, 2026a, p. 33).

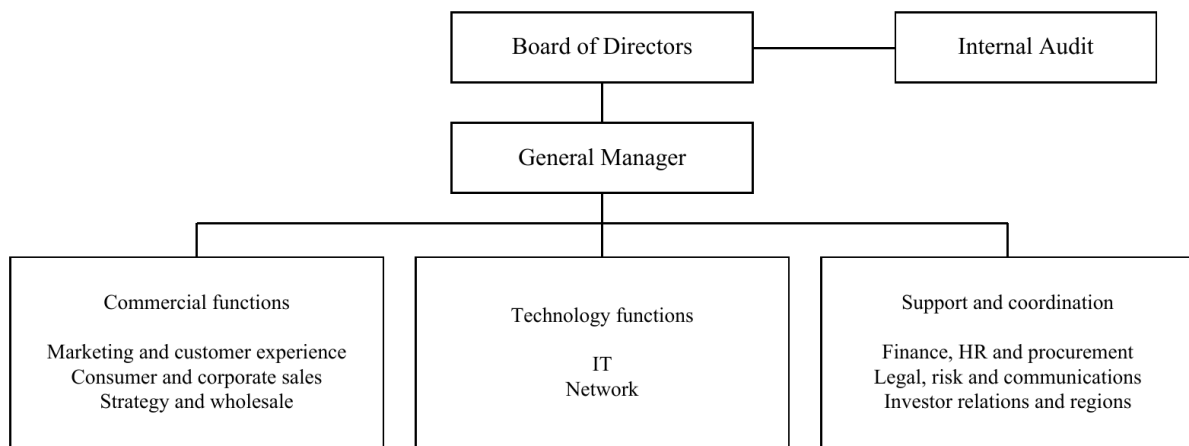


Figure 1. Simplified corporate reporting structure from the 2025 Annual Report.

Network and IT cover communications infrastructure and information systems. Commercial functions include customer experience, consumer sales, corporate sales and wholesale services. Support functions include finance, people operations, procurement, legal compliance and risk. Regional directorates coordinate local operations. My host unit was Planlama ve Geliştirme Müdürlüğü (Planning and Development Directorate). Its formal reporting line remains [TO COMPLETE: department reporting line].

3. Project background

3.1 Department information

My host unit was Planlama ve Geliştirme Müdürlüğü (Planning and Development Directorate). Its specific responsibilities remain [TO COMPLETE: department responsibilities]. My supervisor was Önder Çelebi, whose supplied contact is onder.celebi@turktelekom.com.tr. He is the Director of this unit. I also consulted mentors, whose names, titles and work email addresses remain [TO COMPLETE: mentor details]. These consultations supported an individually conducted project.

The approved arrangement was hybrid, with Monday through Thursday on site and Friday remote. The project record concerns a student research prototype. It does not establish that Türk Telekom already operated this detector, requested a particular production deployment, or used its outputs in customer decisions. Any department-specific business process description requires confirmation before submission.

3.2 Initial project status

The first recorded experiment on 20 July used a small convolutional neural network trained on CIFAKE. The initial workflow was conventional: prepare labelled images, train a classifier, evaluate it and inspect mistakes. A 96.75% result on the familiar test collection initially looked promising, but accuracy fell to 77.1% on a separate 995-image collection. Later auditing also exposed confounds in that external collection. These historical values describe early diagnostics, not a certified deployment benchmark (Keleş, 2026, E1 and E10). Information flowed from dataset sources to local preparation and training scripts, then to saved scores, error analysis and experimental decisions. The work evolved from improving a single accuracy number into controlling what data meant, which populations could influence learning, and how results should reach a user. There was no verified production decision process to replace.

3.3 Motivation and problem definition

The objective was to build and evaluate an image-screening prototype that could find AI-related evidence while limiting false alarms on real photographs. A false positive labels a real image as AI. A false negative misses an AI image. Reducing only one of these errors can make the other worse, so both must remain visible.

The central technical problem was distribution shift: images from a new camera, generator or processing pipeline could behave very differently from the training examples. A separate localization objective required pixel-level evidence about edited regions. An image-level score alone cannot establish that a highlighted area was edited, and a low score cannot prove that an image is authentic.

3.4 Related literature

Wang et al. (2020) studied transfer from a detector trained on one generator to several other synthesis methods. Ojha et al. (2023) showed that pretrained visual representations can support broader fake-image detection with simple classifiers. These studies motivated comparison of learned representations with the early convolutional and handcrafted baselines. Their published results belong to their own protocols and are not PixelProof measurements.

CLIP learns visual representations from image-text supervision (Radford et al., 2021). DINOv2 learns reusable visual features without task-specific labels (Oquab et al., 2024). PixelProof used such pretrained representations and trained adaptation components around them. The contribution is therefore not training an entire foundation model from scratch. It lies in data preparation, learned adaptation, evaluation and integration.

Grommelt et al. (2024) show how compression and image-size differences can become shortcuts in generated-image detection datasets. This matters because matching file extensions or resizing every image does not erase previous processing. PixelProof

investigated processing history and matched transformations, while avoiding claims that cropping automatically removes source bias.

Kim et al. (2026) introduce DEAR, using inpainting-related analysis to prune features that are less robust to processing. PixelProof incorporated a pinned pretrained DEAR component alongside other visual features. A published claim about unseen-generator robustness cannot be inherited merely by loading those weights. Local data lineage and task-specific evaluation remain necessary.

SIDD supplies smartphone denoising data from ten scenes and five cameras (Abdelhamed et al., 2018). SID supplies paired short- and long-exposure low-light raw images (Chen et al., 2018). They provided documented real-image coverage, but neither is a survey of all current phone photography. Multiple observations from a scene remain dependent, and a research raw rendering need not match a consumer camera JPEG.

Guo et al. (2017) distinguish classifier outputs from well-calibrated confidence estimates. That distinction informs the interface: multiplying a score by 100 does not make it a probability of AI generation. Dwork et al. (2015) explain the risk of adaptively reusing holdout data. PixelProof therefore identifies previously inspected collections as development evidence even when individual runs freeze their settings before scoring.

An industry example addresses the same trust problem through a different form of evidence. Adobe Content Credentials can record a file's origin, edits and use of generative AI (Adobe, n.d.). PixelProof instead estimates a signal from image pixels. These approaches could complement each other, but this project did not implement credential verification. The comparison helps explain why a detector score alone should not be presented as proof of authenticity.

4. Internship project

4.1 Project objective and scope

Model1 aimed to detect image-level AI evidence across a wider range of real and generated photographs while preserving detection of AI images already caught by a reference model. Model2 investigated localization of AI edits with ground-truth masks. The practical deliverable was a local student demonstration with clear uncertainty handling, backed by reproducible research records.

The scope included source auditing, training and comparison of candidate models, original-versus-processed evaluation, software validation and readable outputs. It excluded certification of authenticity, unrestricted commercial deployment and a guarantee against all future generators. Classical splicing, generative editing and full-image generation were not treated as interchangeable labels.

4.2 My responsibilities

My project responsibilities covered defining the objectives, directing the experiment sequence, organizing the research material and reviewing the recorded results. The development process used AI coding assistance for implementation, analysis and documentation. Reproducible scripts, source checks and tests were used to verify outputs rather than treating generated explanations as evidence. The report distinguishes those engineering checks from proof that a model generalizes.

I carried out the project individually. I consulted my supervisor and mentors when I needed guidance, but generally made the project decisions myself. They explained evaluation concepts such as true positives and false negatives. Their role was advisory rather than shared implementation. My work covered dataset acquisition and admission records, label mapping, duplicate and ancestry checks, model experiments, API and interface development, and maintenance of PLAN.md, HISTORY.md, ml/EXPERIMENTS.md and DATASETS.md.

Public pretrained encoders and external datasets are acknowledged separately from the project-owned adaptation and application code.

4.3 Methodology and tools

The workflow became more disciplined as problems emerged. In later experiments, PLAN.md stated the question, allowed data and acceptance rules before a run. The experiment log recorded the settings and measurements, including failures. HISTORY.md explained what those results changed, while DATASETS.md recorded acquisition, source checks and permitted use. Earlier entries were retained when a later audit corrected them. This made it possible to trace an idea from its original motivation through its test to the decision that followed.

Training data taught the model. Calibration data helped choose the score threshold. Development data revealed weaknesses and guided further changes. Final evaluation required data that had not influenced these decisions. Repeatedly checking the same collection makes it development evidence, even when each new run has fixed settings. The main error measures were AI detection rate and false alerts on real photographs. AUC measured ranking across thresholds; it did not establish that a particular decision rule was safe.

Python, PyTorch, NumPy, SciPy and scikit-learn supported modelling and analysis. React and TypeScript provided the interface, with FastAPI serving inference. Git preserved changes and automated checks tested the software. Saved models and key inputs were identified by file hashes so that later comparisons could use the intended versions. Table 1 gives representative decisions recorded during this workflow (Keleş, 2026).

Table 1. How recorded findings changed development.

Finding	Recorded response
Weak transfer to new image sources	Compare sources and processing conditions
Reversed source labels in E19b	Remap labels and rerun affected experiments
Evaluation used in E27 threshold selection	Recompute using calibration data only
E92 reduced false alerts but added an AI miss	Retain the failed acceptance result
E151 features harmed processed-image detection	Reject the candidate and investigate its errors

4.4 Expected outcomes and deliverables

The intended outcome was a demonstrable image-screening workflow supported by traceable measurements. Deliverables include trained research artifacts, command-line experiments, a local web demo, dataset provenance records and aggregate evidence files. Model2 contributes an experimental localization evaluator and controlled tests. A production-ready universal detector was an aspiration, not a completed deliverable.

4.5 Project details

4.5.1 Early models and the first transfer failures

The first phase compared a small CNN, classifiers on frozen embeddings and a pretrained ResNet-18. The CNN used 90,000 CIFAKE training images and 10,000 validation images; its familiar test contained 20,000 images. ResNet improved familiar-test accuracy to 97.66%, yet accuracy on the separate 995-image collection fell to 25.2%. Reproducing the training resolution bottleneck recovered much of that loss. These comparisons showed why a larger familiar dataset and a stronger architecture did not establish reliable transfer (Keleş, 2026, E1–E6).

Native-resolution data, image statistics and tile features were then tested to retain fine detail. Some methods helped particular sources but harmed others. Averaging their scores did not consistently improve detection, so attention shifted toward source coverage and evaluation design.

Later audits qualified the early explanations. Fixed-size crops do not remove every dataset shortcut, and plausible camera-noise arguments do not establish what a model learned. These were hypotheses to test, not universal rules.

The E20 tile model was compared with frozen Community-Forensics and B-Free detectors. Each still falsely flagged particular real-image sources. E26 combined source-calibrated alerts, but modern-generator probes found blind spots. E31 and E32 also lost performance on new real sources, showing why the first working demo required further research (Keleş, 2026, E20–E32).

Threshold changes and balanced sources did not consistently solve transfer. E42 failed an RR evaluation of 16,953 images and 50,858 linked views. E43 improved familiar RR results but struggled with reconstructed images. Specialist combinations also missed GAN images or raised real-photo errors. Later correction models therefore had to reduce false alerts while protecting previously detected AI examples (Keleş, 2026, E33–E50).

4.5.2 Data integrity and corrected conclusions

A major audit found that two source datasets used the opposite numeric label convention from the project. PixelProof defines 0 as real and 1 as AI, but raw source labels had entered a shared pool without translation. The error affected several earlier experiments. The remedy was an explicit source-to-project label mapping, checks against declared label names, rebuilt indices and reruns of affected experiments (Keleş, 2026, E19b–E19c).

The corrected labels changed which methods were worth pursuing. DINOv2 had scored AUC 0.480 on Defactify, leading to an explanation that its features were unsuitable. After

correction, AUC rose to 0.764 and that explanation was withdrawn. The claim that a larger training pool brought little benefit also needed revision: it reduced false alerts more than first reported, although performance still varied by source. The correction changed both the measurements and the next research direction (Keleş, 2026, E19c).

A separate E27 audit found that a threshold-selection procedure could consult evaluation data. The corrected procedure used only calibration data before evaluation. The revised candidate then failed its admission requirement and was removed from the serving path. These corrections establish why the provenance of a score matters as much as its size.

4.5.3 Data roles and processing conditions

The dataset register recorded each source, its licence, intended use and integrity checks. Acquired archives, selected training images and evaluation images were distinct populations. Download size therefore did not measure how many examples trained E92 or how reliable it was. The audit checked identical files, similar images and known shared scenes or prompts. Where origins remained unknown, that uncertainty was retained. A parent image means an original observation before this project creates resized or compressed versions of it.

E92 used 12,269 training parent images: 7,674 real and 4,595 AI. Four processing conditions produced 49,076 views. Later research added real images, giving 12,525 parents and 50,100 views; these were not the earlier E92 training counts. Table 2 places the E92 comparison alongside larger earlier evaluations. Each row describes a different model or data role, and the rows must not be summed into one independent test (Keleş, 2026, E42, E49, E92 and E139).

Table 2. Training and evaluation populations at different project stages.

Population	Parents	Views	Role
E42 RR evaluation	16,953	50,858	Earlier model; failed acceptance
E43 on E49-C	2,000	4,000	Earlier final; failed acceptance
E92 training	12,269	49,076	Training
E92 on E66	320	640	Reused development
Later source-fold research	12,525	50,100	Internal diagnostics
Model2 controlled pilot	16	192 in E135	Adaptive development

The four training conditions were clean input, an assigned transport transformation, JPEG quality 75 and a social-style transformation that limited the longest side to 1080 pixels before JPEG quality 75 encoding. The development comparison used original and social-style views of the same 320 parents. More transformed views improve coverage of a processing question, but do not create more independent photographs.

An earlier, larger E49-C test contained 1,000 real and 1,000 AI parent images. E43 falsely flagged 391 real originals and 490 processed copies, while detecting 943 and 955 AI images respectively. It passed 11 of 20 checks. After inspection, these results could guide research but no longer serve as an untouched final test. E92 was not evaluated on this collection because it failed the earlier acceptance stage. Its later 20/20 development result therefore cannot be reported as passing E49-C (Keleş, 2026, E49 and E92).

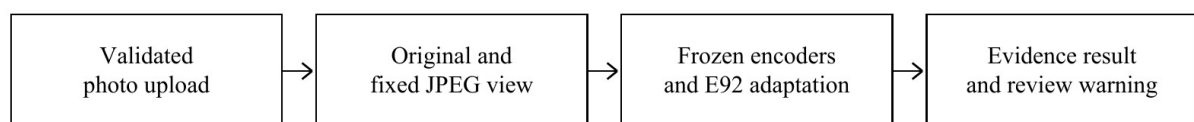
4.5.4 E92 representation and learning

The route to E92 involved several rejected alternatives. Broader real-image coverage reduced false alerts but sometimes lost AI detections. Fine-tuning, grayscale input and small score corrections did not meet the full requirements. Later work added CLIP and DEAR features, a learned representation, and processing conditions closer to web use. E83 and E86 each passed

17 of 20 development checks. Additional training coverage preceded E92, although this sequence alone does not prove which added source caused the improvement (Keleş, 2026, E51–E92).

E92 uses pretrained DINOv2, CLIP and DEAR components to turn images into numerical features. Their weights stay fixed. Project-trained components then learn how to use those features, and a correction adjusts the reference score. The correction was constrained to protect AI responses on training examples while reducing errors on difficult real-image sources. This protection had to be checked again on development images; a training constraint alone could not guarantee it.

Figure 2 shows the application-level information flow. It deliberately separates frozen pretrained components, project adaptation and the interface decision. The browser receives a validated response rather than fitting a model. The local service verifies required artifacts and uses the original image plus a fixed processed view. No file name or camera declaration is an automatic authenticity rule.



Raw model scores remain separate from authenticity claims.

Figure 2. PixelProof inference flow and separation of model evidence from display policy.

This architecture is a composite research system. Referring to it as one downloadable universal model would conceal its dependencies, provenance limits and licensing conditions. E92 is the identifier of a frozen candidate, not a percentage score or a count of training epochs.

4.5.5 Development improvement and acceptance limits

Figure 3 compares false alerts on exactly the same real development images. E92 reduced original-view false alerts from 68 of 160 for E43 to zero, and processed-view false alerts from 68 to 14. E86 was the preceding intermediate candidate. The comparison is paired and descriptive; it is not an independent estimate for all future photographs.

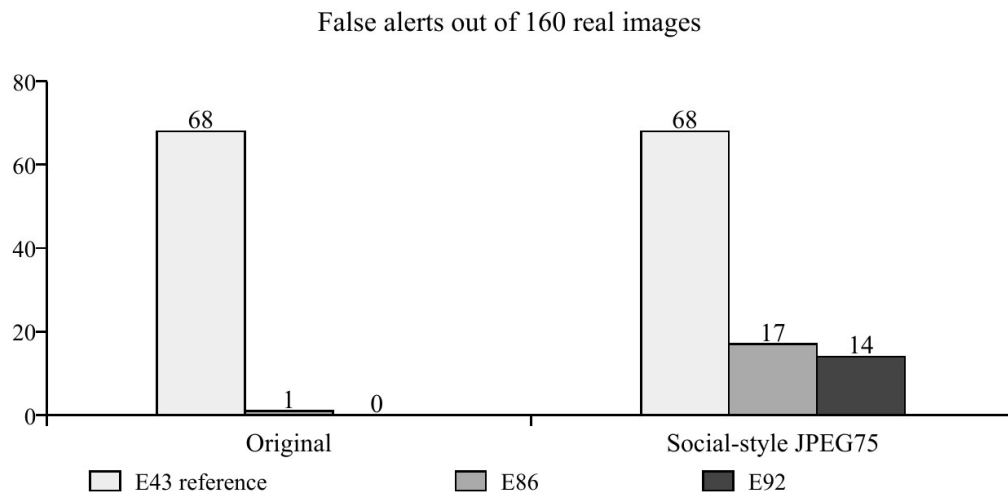


Figure 3. Real-image false alerts on the reused development set, with 160 real images per condition.

E92 detected 159 of 160 AI images in each condition. It passed ten numerical requirements for originals and the same ten for processed views. However, it newly missed one original AI image that E43 had caught. Other recovered AI images could not cancel that loss under the predeclared retention rule. Consequently, the full set of acceptance requirements failed despite the 20/20 numerical milestone.

The development collection contains 160 real observations from only ten SIDD scenes and 160 AI images from two previously seen families. Repeated use influenced later research choices. The result therefore supports progress on this collection, while leaving adaptive selection bias and unseen-source reliability unresolved. Thirteen of the fourteen processed real false alerts concentrate in two scenes, which also shows why a pooled rate can conceal weaknesses.

The later E102 candidate reduced processed-view real false alerts further, from 14 to 12 of 160, and retained every E92-detected AI image. Nevertheless, it still missed the same E43-detected original AI example and failed complete acceptance. It was not promoted, and its gallery and E49 evaluations were not opened. E92 therefore identifies the current demo version, not the lowest value ever recorded in a development table (Keleş, 2026, E102).

4.5.6 Honest scores and the web demonstration

The demo reports AI evidence, no clear AI evidence or uncertainty. It cannot certify authenticity. On its 0–100 score scale, the upper threshold is about 7.94 and the lower about 1.15. The upper came from E48 calibration with real-photo false-alert limits; the lower came from E49 analysis of already-used calibration and development data. These are experimental operating points, not probabilities or universal optima.

An original score at or above the upper threshold keeps an AI alert visible, with a review warning if needed. Both E92 views below the lower threshold yield no clear AI evidence; remaining cases are uncertain. E43 provides a separate advisory and cannot veto two low E92 scores. Decisions use unrounded scores rather than a 50% boundary.

The combined rule was checked on the same 320 development images. Among 160 real images, 139 received no clear AI evidence, 21 were uncertain and none received an AI alert. Among 160 AI images, 159 received an alert and one was uncertain. Thus 14 processed-view false alerts do not mean 14 final website warnings: a low original score can leave the combined result uncertain. These reused observations do not predict every future upload (Keleş, 2026, current-policy audit).

The service rejects unsupported or oversized uploads, checks model files before loading and reports unavailable models explicitly. Cancellation handling prevents overlapping heavy inference jobs, and the browser validates response fields. These repairs improve reliable operation; they do not increase measured detection accuracy.

4.5.7 Broader checks and rejected improvements

The owner-gallery check found 9 false AI alerts among 206 distinct original real-image files, rising to 18 after social-style processing: 4.37% and 8.74%. The gallery mostly represents one phone and related scenes. It had already influenced earlier development and calibration decisions, so it is not an independent test. It nevertheless shows practical mistakes that the main development result could hide. Private photographs and individual file identifiers are excluded from this report (Keleş, 2026, E95).

Later research compared fresh diagnostic heads using source-separated folds. E146 tested calibration on one source fold and evaluation on another. Three of six calibration assignments were feasible, but no assignment met the full evaluation conditions. These were separate diagnostic models, not changes to E92. The result rejects that tested calibration recipe rather than proving that all calibration is impossible.

E148 found a processing imbalance in the existing training collection: known 224-pixel derivatives represented 29.18% of real images and 7.68% of AI images. Such differences are possible shortcuts, not evidence of a specific causal mechanism. E150B successfully extracted new source-pixel features for all 12,525 parents after a memory-safe implementation amendment.

The added features slightly improved clean-image averages but reduced AI detection after all three tested processing conditions. In one JPEG75 comparison, detection fell from 75.73% to 74.15%. A follow-up analysis examined 131 newly missed AI views. The added residual feature term was the largest harmful score change in 94 cases. This helped locate a weakness, but did not prove that removing the term would fix it. The candidate was rejected (Keleş, 2026, E151–E152).

The later training audit could form only three AI-bearing groups under the known source relationships. Some shared origins and prompts remained unknown. Another 300 AI images

in MNW and 100 real images in HDR+ remained unscored, but they had not been approved together as a balanced, independent final test. Unused data are useful only if their coverage and independence answer the intended question (Keleş, 2026, E139–E146).

4.5.8 Model2 localization and its limits

Model2 compares predicted edit regions with reference masks and checks authentic images for false alarms. Early tests used the true mask size to choose the predicted area, which made their overlap scores misleading. In the later 120-image CocoGlide evaluation, a simple central-region baseline still outperformed the detector. This prompted tests with edits placed elsewhere. A separate audit also found shared COCO image origins, limiting claims of independent data (Keleş, 2026, E17, E107 and E115–E117).

The controlled pilot used sixteen previously inspected images and one Stable Diffusion 1.5 editor. Moving edits reduced the earlier model’s pixel AUC from 0.742 to 0.566. Training on both placements partly recovered new-placement performance, but harmed original-placement performance and highlighted more authentic pixels incorrectly. Table 3 summarizes these trade-offs, with AUC rounded to two decimals (Keleş, 2026, E132–E135).

Table 3. Model2 placement sensitivity on sixteen reused parents, original-image condition.

Endpoint	Earlier head	Two-placement head
Original-placement pixel AUC	0.74	0.70
New-placement pixel AUC	0.57	0.64
Authentic falsely flagged area	17.11%	25.74%

The candidate was rejected because improvements introduced other errors. Moving masks also changes edited content, so this is not pure causal proof of location bias. Model2 remains experimental and needs independent parents and more editors.

4.6 Results

The project delivered a trained E92 research system, a local web demonstration and a reproducible record of model development and rejection. Table 4 reports the central E92 per-view outcome at the fixed upper cut. The parent collection is reused development data, with 160 real and 160 AI images per condition. These rates do not describe the paired interface policy.

Table 4. E92 per-view results on the reused E66 development collection.

Measure	Original	Social-style
AI detection	159/160 (99.38%)	159/160 (99.38%)
Real false alerts	0/160 (0.00%)	14/160 (8.75%)
Balanced accuracy	99.69%	95.31%
Numerical criteria	10/10	10/10
New AI misses versus E43	1	0

The 20 numerical checks passed, but full acceptance failed because of the one new original AI miss. Universal reliability and independent final-test success were not achieved. Model2 did not qualify as a reliable localization service. The latest recorded engineering checkpoint passed 1,245 Python tests; this measures software checks, not 1,245 successful image classifications. There is no measured company deployment impact or production rollout commitment. The completed outcome is a student research prototype with documented limitations (Keleş, 2026).

5. Internship experience

5.1 Learning

The most useful lesson from the work is to ask what a good score actually proves. The first high accuracy values encouraged further development, but errors on new sources changed the priority. The project then needed better data checks and comparisons, not just a larger model. The reversed-label incident is the clearest example: checking the inputs overturned an explanation that had sounded convincing.

I learned how to sustain a long project on my own and reduce its scope when the work became too complex. Asking focused questions helped me communicate problems and use advice while remaining responsible for my decisions. Running my own experiments made machine learning concepts more concrete. I was already interested in this field before the internship; applying it in a substantial project strengthened that interest and gave me a clearer understanding of the work involved.

5.2 Relation to undergraduate education

CS412 Machine Learning gave me the concepts I used to start this project and develop the initial experiments. PROJ201 had introduced me to applied machine learning through a team project for an application intended to guide visually impaired users. We trained YOLOv11 on a custom dataset and integrated it into the application; our team placed first in our group. That experience gave me an initial understanding of model training and application integration. DSA210 provided my early foundations in data science, which helped me approach the data preparation and analysis in this internship.

The internship exposed preparation needs beyond the initial model fit. Experimental design and data provenance required particular attention: valid image files could still have inconsistent labels, and different files could share scenes or generation ancestry. These issues became clearer through the project's failed experiments and corrections.

5.3 Major difficulties

The first difficulty was unreliable transfer across sources. Strong internal scores did not prevent false alarms on unfamiliar real photographs. The response was to expand documented source coverage, report errors by group and distinguish training, calibration and evaluation. The generalization problem remains partly unresolved, so the report records the response without claiming a complete solution.

The second difficulty was trustworthy measurement. Reversed source labels and an evaluation-dependent threshold procedure changed the meaning of earlier results. Explicit label mapping, isolated calibration and reruns corrected the affected evidence. The history retained both original and corrected conclusions, making the correction auditable.

The third difficulty was running a multi-component research system within local resource and reliability limits. Feature caching, bounded batches and resource guards helped. An E150 memory failure led to a separately recorded, pixel-equivalent implementation amendment, E150B. Runtime checks addressed cancellation and artifact-manifest handling without changing the scientific model. These are concrete engineering repairs rather than proof of better detection.

5.4 A typical day

The approved schedule was Monday through Thursday on site and Friday remote. On office days, I usually arrived at 7:30 a.m. After breakfast, I worked through the morning until around noon or early afternoon. I continued working afterwards, with occasional short tea or coffee breaks.

My tasks followed the stage of the project: reviewing the plan, implementing a change, running experiments, interpreting results and updating the records. When a question arose, I consulted my supervisor or mentors for clarification and then decided how to proceed. This routine helped me maintain progress during a long individual project.

6. Conclusions

PixelProof progressed from early classifiers with large source-transfer failures to a working research demo with strong results on a defined development collection. E92 reduced real-image false alerts while maintaining high aggregate AI detection in that collection. The interface now distinguishes a model signal from an authenticity claim and separates the main model from an advisory reference.

The main result has a clear limit. All 20 numerical checks passed, but one AI image caught by the reference became a new miss. Reusing the development collection also limits claims about new sources. Later source tests and the Model2 pilot exposed other weaknesses. Keeping these failures in the records helped explain why candidates were rejected and why the demo remained a research prototype.

The durable outcome is both an implemented prototype and a traceable method of evaluating it. Data corrections, rejected hypotheses and runtime repairs are part of that outcome. They support a credible student engineering project while identifying exactly which claims need further evidence.

7. Recommendations

Future internship students should establish dataset meanings, permissible data roles and error metrics before a long training run. They should learn to reproduce a saved result and inspect individual changes, since a better average can hide newly harmed examples. Basic familiarity with version control, Python testing and the host organization's data rules would reduce avoidable delays.

For this project, the next research priority is an explicitly independent and sufficiently diverse evaluation design. Source ancestry, scene dependence and generator coverage should be resolved before claiming independence. Calibration must remain separate from evaluation, and operating costs should determine a prospectively chosen decision policy. Model2 needs more independent parents and editors together with authentic and conventional-edit controls.

In the workplace, students should agree on scope and review expectations with their supervisor, record incomplete work honestly and protect personal or restricted data. Results should be communicated in plain language: what changed, which population supports the finding and what still fails. AI coding assistance can accelerate work, but the student must understand and verify what will be submitted and presented.

8. References

- Abdelhamed, A., Lin, S., & Brown, M. S. (2018). A high-quality denoising dataset for smartphone cameras. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. Retrieved September 25, 2026, from <https://abdokamel.github.io/sidd/>
- Adobe. (n.d.). Content Credentials overview. Adobe Help Center. Retrieved September 25, 2026, from <https://helpx.adobe.com/creative-cloud/apps/adobe-content-authenticity/content-credentials/overview.html>
- Chen, C., Chen, Q., Xu, J., & Koltun, V. (2018). Learning to see in the dark. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. Retrieved September 25, 2026, from <https://cchen156.github.io/SID.html>
- Dwork, C., Feldman, V., Hardt, M., Pitassi, T., Reingold, O., & Roth, A. (2015). Generalization in adaptive data analysis and holdout reuse. *Advances in Neural Information Processing Systems*, 28. Retrieved September 25, 2026, from <https://proceedings.neurips.cc/paper/2015/hash/bad5f33780c42f2588878a9d07405083-Abstract.html>
- Grommelt, P., Weiss, L., Pfreundt, F.-J., & Keuper, J. (2024). Fake or JPEG? Revealing common biases in generated image detection datasets. arXiv:2403.17608. Retrieved September 25, 2026, from <https://arxiv.org/abs/2403.17608>
- Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *Proceedings of Machine Learning Research*, 70, 1321–1330. Retrieved September 25, 2026, from <https://proceedings.mlr.press/v70/guo17a.html>
- Keleş, E. H. (2026). PixelProof experiment, history and dataset records [Software and research records, commit 497d45c]. GitHub. Retrieved September 25, 2026, from <https://github.com/EfeHanKeles346/ai-image-detector/tree/497d45c>
- Kim, D., Choi, J., Seong, H. S., Kim, S., Lee, D., Yi, S., & Choi, J.-H. (2026). Dissect and prune: Enhancing robustness in AI-generated image detection. ICML 2026; arXiv:2606.10309, version 2. Retrieved September 25, 2026, from <https://arxiv.org/abs/2606.10309v2>
- Ojha, U., Li, Y., & Lee, Y. J. (2023). Towards universal fake image detectors that generalize across generative models. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 24480–24489. Retrieved September 25, 2026, from <https://arxiv.org/abs/2302.10174>
- Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.-Y., Li, S.-W., Misra, I., Rabbat, M., Sharma, V., Synnaeve, G., Xu, H., Jégou, H., Mairal, J., Labatut, P., Joulin, A., & Bojanowski, P. (2024). DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*. Retrieved September 25, 2026, from <https://arxiv.org/abs/2304.07193>

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. *Proceedings of Machine Learning Research*, 139, 8748–8763. Retrieved September 25, 2026, from <https://proceedings.mlr.press/v139/radford21a.html>

Türk Telekom. (2026a). 2025 annual report. Türk Telekom Investor Relations. Retrieved September 25, 2026, from <https://www.ttyatirimciiliskileri.com.tr/media/dqsjlvxo/2025-annual-report.pdf>

Türk Telekom. (2026b). Hakkımızda [About us; operating figures at June 30, 2026]. Retrieved September 25, 2026, from <https://www.turktelekom.com.tr/hakkimizda/detay?p=tab3>

Wang, S.-Y., Wang, O., Zhang, R., Owens, A., & Efros, A. A. (2020). CNN-generated images are surprisingly easy to spot... for now. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 8695–8704. Retrieved September 25, 2026, from <https://arxiv.org/abs/1912.11035>

9. Appendices

9.1 Development criteria

Table 5 lists the ten inherited numerical criteria, applied separately to original and social-style views. They are project-defined requirements, not an external certification standard. Automatic coverage and uncertainty are complementary, and several other metrics are correlated. The separate rule against newly missing any reference-detected AI image failed for E92. Coverage and uncertainty here describe each view separately; they are not the paired website outcomes in Section 4.5.6.

Table 5. E92 numerical development checks and their measured values.

Criterion	Requirement	Original	Social
Valid-score coverage	100%	100%	100%
ROC-AUC	≥ 0.90	0.9998	0.9942
Balanced accuracy	$\geq 85\%$	99.69%	95.31%
Pooled real false alerts	$\leq 10\%$	0.00%	8.75%
Worst real source	$\leq 20\%$	0.00%	18.75%
Pooled AI detection	$\geq 80\%$	99.38%	99.38%
Worst AI source	$\geq 60\%$	98.75%	98.75%
Automatic coverage	$\geq 80\%$	98.75%	98.44%
Accuracy when automatic	$\geq 95\%$	99.68%	95.24%
Uncertain fraction	$\leq 20\%$	1.25%	1.56%

9.2 Evidence and reproduction map

Table 6 maps the principal findings to compact repository evidence. The source snapshot is commit 497d45c. Large weights, datasets and private predictions remain outside Git. Reproducing an existing receipt is distinct from training a new candidate or opening a protected final collection.

Table 6. Primary project evidence for the report.

Finding	Repository evidence
Earlier comprehensive-final failure	evidence/e49_final_result.json
E92 numerical result and failed retention	evidence/e92_development.json
Unpromoted E102 successor	evidence/e102_development.json
Bias and readiness audit	evidence/e92_readiness_audit_2026-09-14.json
Current paired interface policy	evidence/project_audit_20260916.json
Owner-gallery aggregate results	evidence/e95_gallery_report.json
Model2 placement comparison	evidence/e135_location_learning.json
Processing imbalance	evidence/e148_processing_inventory.json
Residual-feature rejection	evidence/e151_source_pixel_comparison.json
Fixed-model score decomposition	evidence/e152_logit_decomposition.json

AI recall is TP divided by TP plus FN. Real false-positive rate is FP divided by FP plus TN. Balanced accuracy averages AI recall and real specificity. Automatic coverage is the fraction receiving an automatic decision; accuracy among those decisions excludes abstentions from its denominator. Pixel AUC ranks edited against unedited locations, while intersection-over-union measures overlap at a chosen map threshold. These quantities answer different questions and should not be exchanged.